CSE4203: Computer Graphics

Lecture – 2

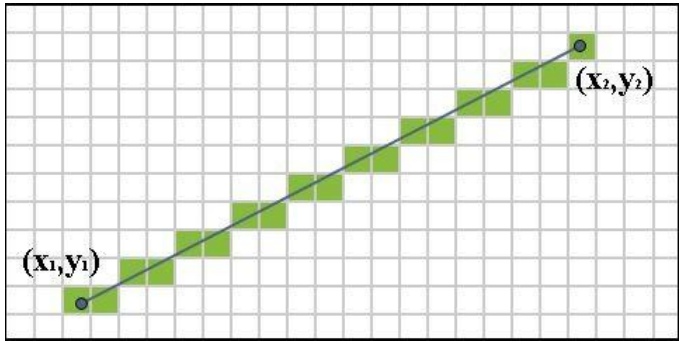
Raster Graphics

Outline

- Raster and Raster Images
- Image Compression
- Display Devices
- Pixel Values
- RGB Color
- Graphical Processing Unit

Raster (1/1)

- Most computer graphics images are presented on *raster display*.
  - i.e. television
  - has rectangular array of small light-emitting **pixels**
    - individually set to different colors to create desired image.

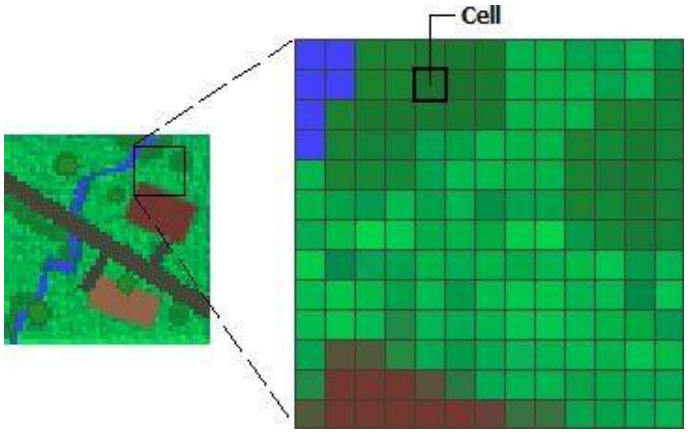


Credit: Fundamentals of Computer Graphics 3rd Edition by Peter Shirley, Steve Marschner | http://www.cs.cornell.edu/courses/cs4620/2019fa/ Image source: https://commons.wikimedia.org/wiki/File:Bresenham_line.png

Image Source: Internet

Storing Images (1/2)

- Raster Image:
  - used to store and process images, as rasters are common in devices
    - simply a **2D array**
    - stores the pixel value foreach pixel
    - usually a color stored as **three numbers (r, g, b)**



Credit: Fundamentals of Computer Graphics 3rd Edition by Peter Shirley, Steve Marschner | http://www.cs.cornell.edu/courses/cs4620/2019fa/

Image source: https://desktop.arcgis.com/en/arcmap/10.3/manage-data/raster-and-images/what-is-raster-data.htm

Image Source: Internet

Storing Images (2/2)

- Raster Image:
  - Considered as device-independent of the image to be displayed
  - Resolution Dependent i.e quality are measured using number of pixels per unit such as DPI
  - Resizing can result quality degradation
  - Common formats are JPEG, PNG, BMP

Credit: Fundamentals of Computer Graphics 3rd Edition by Peter Shirley, Steve Marschner | http://www.cs.cornell.edu/courses/cs4620/2019fa/

Image source: https://desktop.arcgis.com/en/arcmap/10.3/manage-data/raster-and-images/what-is-raster-data.htm

Image Source: Internet

Image Compression (1/3)

- Image compression is a method used to reduce the size of images,
- Improves the rendering speed with reduced file size
- 2 methods of compression:
  - Lossless Compression
  - Lossy Compression

Credit: Lossy vs Lossless Compression: Differences & Advantages | Adobe

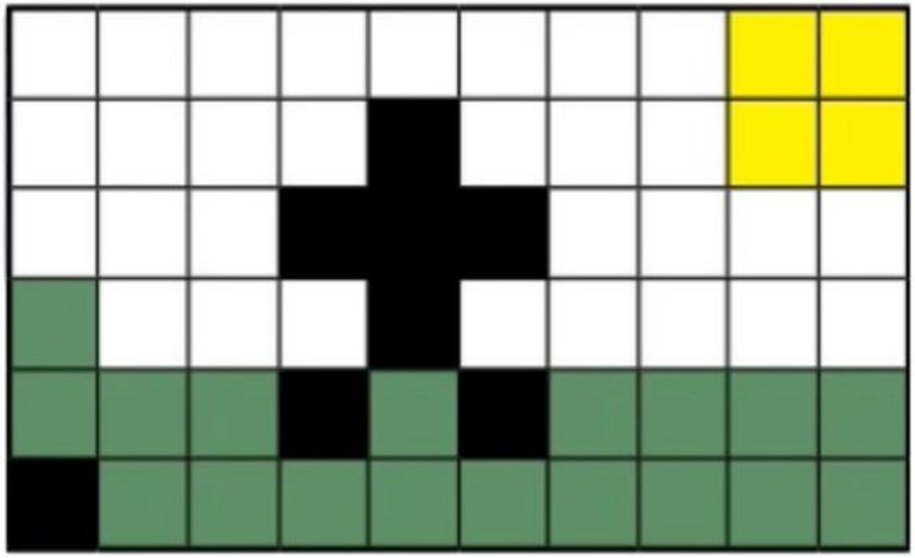
Image Compression (2/3)

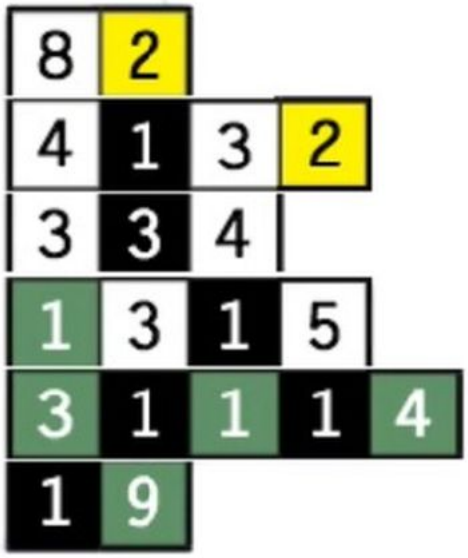
Lossless Compression

- Don’t impact the image quality
- Only removes additional, non-essential data automatically added by the device used to take the photo
- No significant reduction in file size
- Lossless Compression algorithms: Run-length encoding, Huffman coding
- Lossless formats are .RAW, .BMP, .GIF, and .PNG

Credit: Lossy vs Lossless Compression: Differences & Advantages | Adobe

How Run-Length Encoding (RLE) Works





This can be stored as data pairs for example

8W2Y
4W1B3W2Y
3W3B4W
1G3W1B5W
3G1B1G1B4G
1B9G

Credit: https://www.youtube.com/watch?v=cAAeSn8_aCU&ab_channel=MizComputing

Image Compression (3/3)

Lossy Compression

- Reduces the file size considerably by removing image data
- Quality might degrade
- This process is irreversible - can’t get back to the original file
- Common algorithms - discrete wavelet transform, fractal compression, transform encryption etc.
- Lossy format; JPEG, MPEG, AVC

Credit: Lossy vs Lossless Compression: Differences & Advantages | Adobe

Discrete Wavelet Transform

- DWT converts the image into the frequency domain instead of pixel value (spatial domain)

It divides image into four sub-bands:

Low-Low Approximation of image	**Low-High** Contains horizontal edge details
High-Low Contains vertical edge details	**High-High** Contains diagonal details and noise

Credit: Lossy vs Lossless Compression: Differences & Advantages | Adobe

Discrete Wavelet Transform

Given an 2x4 image, we will haar DWT

Original Image:

10	10	20	20
12	12	18	18

Haar DWT pairwise formula:

For a pair (x_0, x_1),

$$a = (x_0 + x_1) / 2$$

$$d = (x_0 - x_1) / 2$$

Discrete Wavelet Transform

10	10	20	20
12	12	18	18

Step-1: Row-wise haar transform:

For each row wise pairs find a and d

Pair (10, 10): a = (10 + 10) / 2 = 10, d = (10 - 10) / 2 = 0

Pair (20, 20): a = (20 + 20) / 2 = 20, d = (20 - 20) / 2 = 0

Pair (12, 12): a = (12 + 12) / 2 = 12, d = (12 - 12) / 2 = 0

Pair (18, 18): a = (18 + 18) / 2 = 18, d = (18 - 18) / 2 = 0

Discrete Wavelet Transform

10	20	0	0
12	18	0	0

Replace the values

Step-1: Row-wise haar transform:

For each row wise pairs find a and d

Pair (10, 10): a = (10 + 10) / 2 = 10, d = (10 - 10) / 2 = 0

Pair (20, 20): a = (20 + 20) / 2 = 20, d = (20 - 20) / 2 = 0

Pair (12, 12): a = (12 + 12) / 2 = 12, d = (12 - 12) / 2 = 0

Pair (18, 18): a = (18 + 18) / 2 = 18, d = (18 - 18) / 2 = 0

Discrete Wavelet Transform

10	20	0	0
12	18	0	0

Step-2: Column-wise Haar transform:

For each column wise pairs find a and d

Pair (10, 12): a = (10 + 12) / 2 = 11, d = (10 - 12) / 2 = -1

Pair (20, 18): a = (20 + 18) / 2 = 19, d = (20 - 18) / 2 = 1

Pair (0, 0): a = 0, d = 0

Pair (0, 0): a = 0, d = 0

Discrete Wavelet Transform

11	19	0	0
-1	1	0	0

Replace the values

Step-2: Column-wise Haar transform:

For each column wise pairs find a and d

Pair (10, 12): a = (10 + 12) / 2 = 11, d = (10 - 12) / 2 = -1

Pair (20, 18): a = (20 + 18) / 2 = 19, d = (20 - 18) / 2 = 1

Pair (0, 0): a = 0, d = 0

Pair (0, 0): a = 0, d = 0

Discrete Wavelet Transform

11	19	0	0
-1	1	0	0

Step-3: Thresholding

Let threshold = 2,

Any coefficient with value < 2 becomes 0

11	19	0	0
0	0	0	0

Compress format after DWT

Reconstruct: Inverse DWT

11	19	0	0
0	0	0	0

Step-1: Inverse Column - For each column pair (A,D)

$$x_0 = A + D$$

$$x_1 = A - D$$

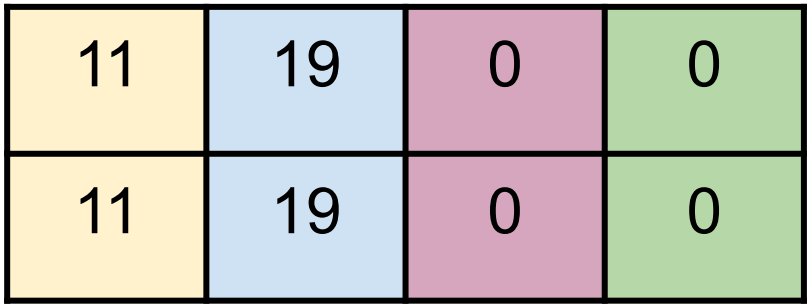
Column 1: A = 11, D = 0, $x_0 = 11 + 0 = 11$, $x_1 = 11 - 0 = 11$

Column 2: A = 19, D = 0, $x_0 = 19 + 0 = 19$, $x_1 = 19 - 0 = 19$

Column 3: (0, 0)

Column 4: (0, 0)

Reconstruct: Inverse DWT



Step-1: Inverse Column

Replace the values

Column 1: A = 11, D = 0, $x_0 = 11 + 0 = 11$, $x_1 = 11 - 0 = 11$

Column 2: A = 19, D = 0, $x_0 = 19 + 0 = 19$, $x_1 = 19 - 0 = 19$

Column 3: (0, 0)

Column 4: (0, 0)

Reconstruct: Inverse DWT

11	19	0	0
11	19	0	0

Step-1: Inverse Row - For each row pair value (A, D)

$$x_0 = A + D$$

$$x_1 = A - D$$

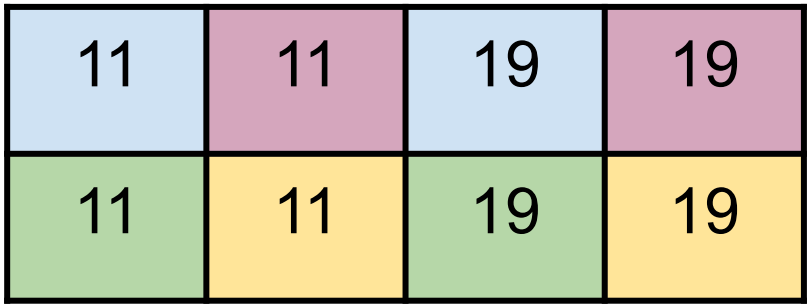
Row 1:

Pair: A = 11, D = 0, $x_0 = 11 + 0 = 11$, $x_1 = 11 - 0 = 11$

Pair: A = 19, D = 0, $x_0 = 19 + 0 = 19$, $x_1 = 19 - 0 = 19$

Row 2: (11, 11), (19, 19) [Same]

Reconstruct: Inverse DWT



Step-1: Inverse Row

Replace the values

Row 1:

Pair: A = 11, D = 0, $x_0 = 11 + 0 = 11$, $x_1 = 11 - 0 = 11$

Pair: A = 19, D = 0, $x_0 = 19 + 0 = 19$, $x_1 = 19 - 0 = 19$

Row 2: (11, 11), (19, 19) [Same]

Reconstruct: Inverse DWT

10	10	20	20
12	12	18	18

Original Image

11	11	19	19
11	11	19	19

Reconstructed Image

So, the pixels changed by small margin — small blur/smoothing occurs.

Image Compression (4/4)



Image Source: Internet

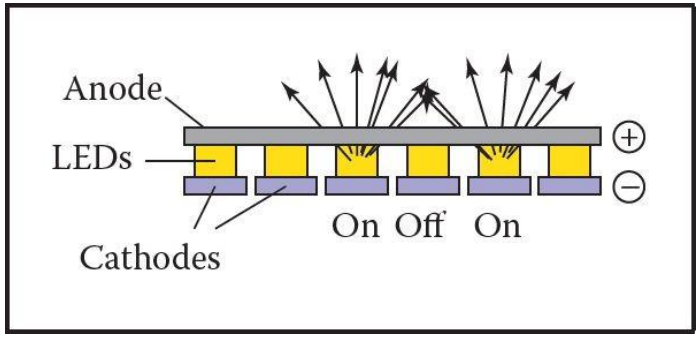
Display Devices (1/1)

- Transmissive Displays:
  - require a light source to illuminate them
  - backlight behind the array
    - i.e. in a projector, a lamp emits light projected onto the screen after passing through the array.
- Emissive Display:
  - it is its own light source.

Credit: Fundamentals of Computer Graphics 3rd Edition by Peter Shirley, Steve Marschner | http://www.cs.cornell.edu/courses/cs4620/2019fa/

Emissive Displays (1/3)

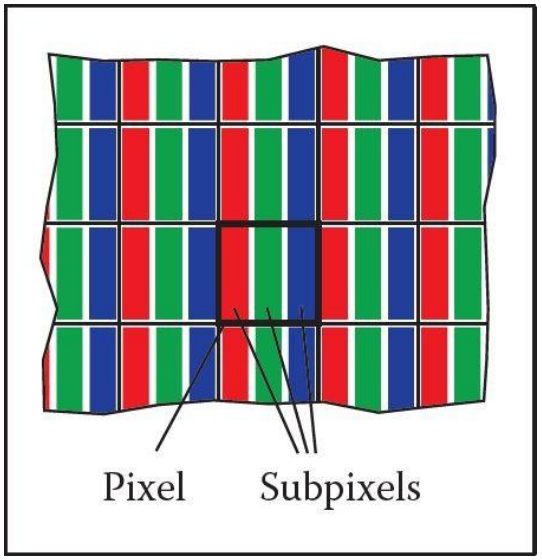
- Emissive Displays:
  - Example: light-emitting diode (LED)
  - Each pixel is composed of one or more LEDs (semiconductor devices)
    - emit light with intensity ↔ electrical current passing through them



Credit: Fundamentals of Computer Graphics 3rd Edition by Peter Shirley, Steve Marschner | http://www.cs.cornell.edu/courses/cs4620/2019fa/

Emissive Displays (2/3)

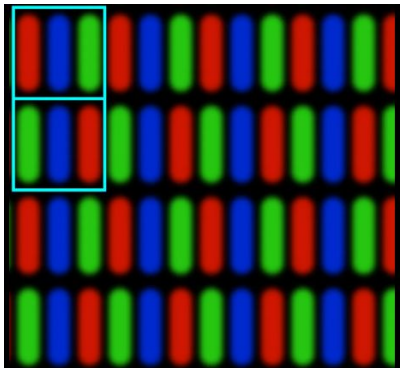
- Sub-pixel:
  - —Pixels divided into three independently controlled sub-pixels (R, G, B)
    - each with own LED (different materials)
    - emit light of different colors



Credit: Fundamentals of Computer Graphics 3rd Edition by Peter Shirley, Steve Marschner | http://www.cs.cornell.edu/courses/cs4620/2019fa/

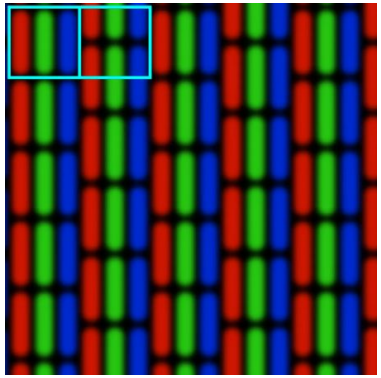
Emissive Displays (2/3)

Sub-pixel Orientations



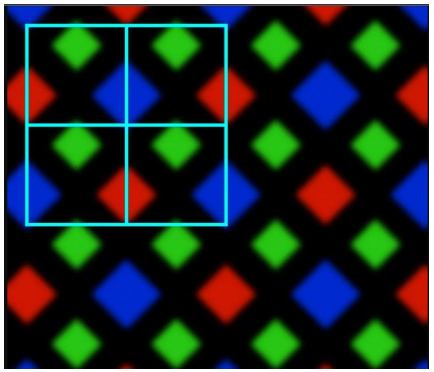
Vertical Stripe

- Most Common
- LCD monitors
- Image pixel directly maps to sub-pixels



Triangular (Delta)

- Triangle clusters
- Not Aligned linearly
- CRT phosphor monitors



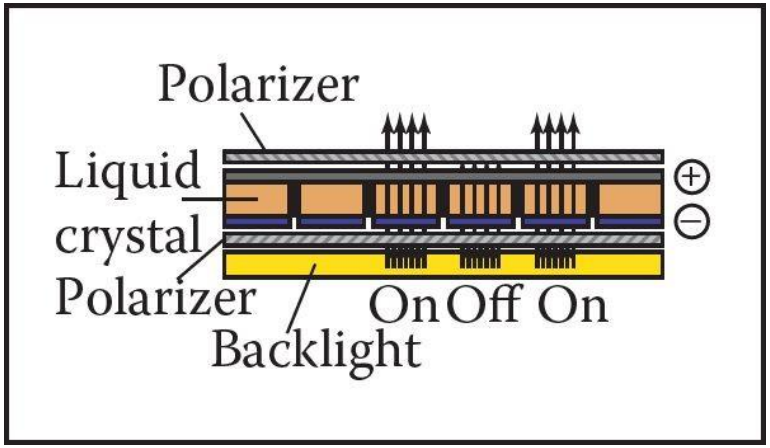
PenTile Matrix

- Have more green subpixels than red and blue
- Need fewer subpixels, energy efficient
- Mobile OLED/AMOLED Displays

Source: https://geometrian.com/resources/subpixelzoo/

Transmissive Displays (1/3)

- Transmissive Displays:
  - Example: light crystal display (LCD)

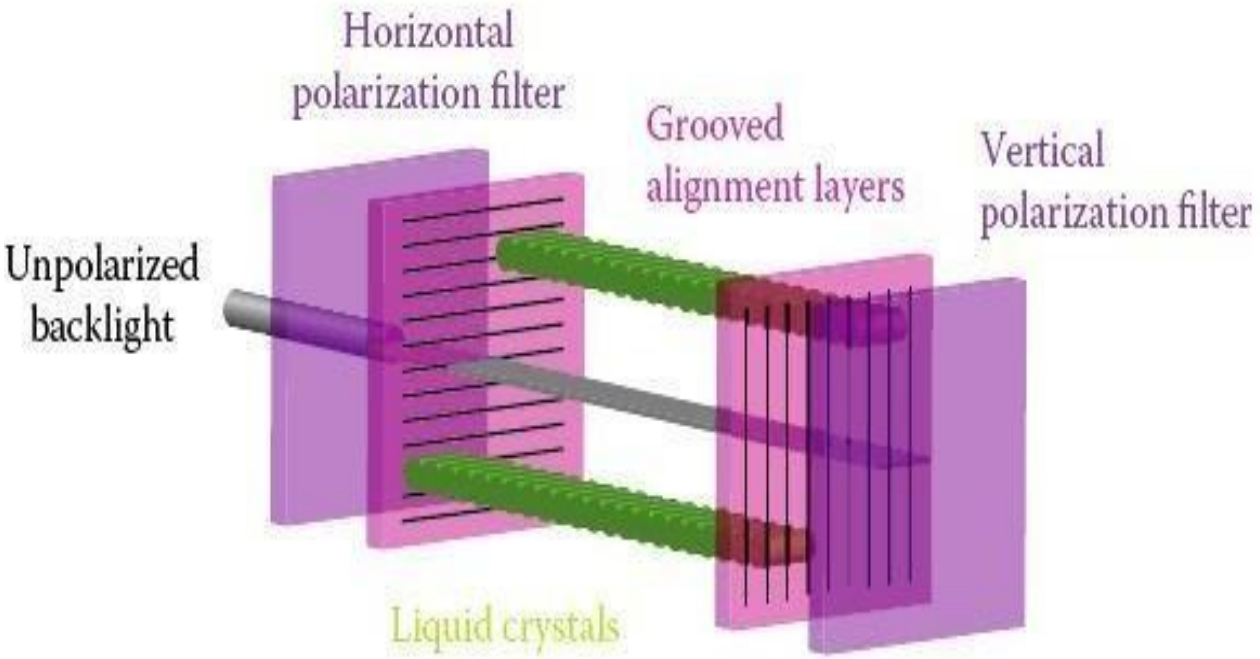


- Molecular structure of liquid crystal rotates the polarization of light that passes through it
- LCDs also have sub-pixels.

Credit: Fundamentals of Computer Graphics 3rd Edition by Peter Shirley, Steve Marschner | http://www.cs.cornell.edu/courses/cs4620/2019fa/

Transmissive Displays (2/3)

- Degree of rotation ↔ applied voltage

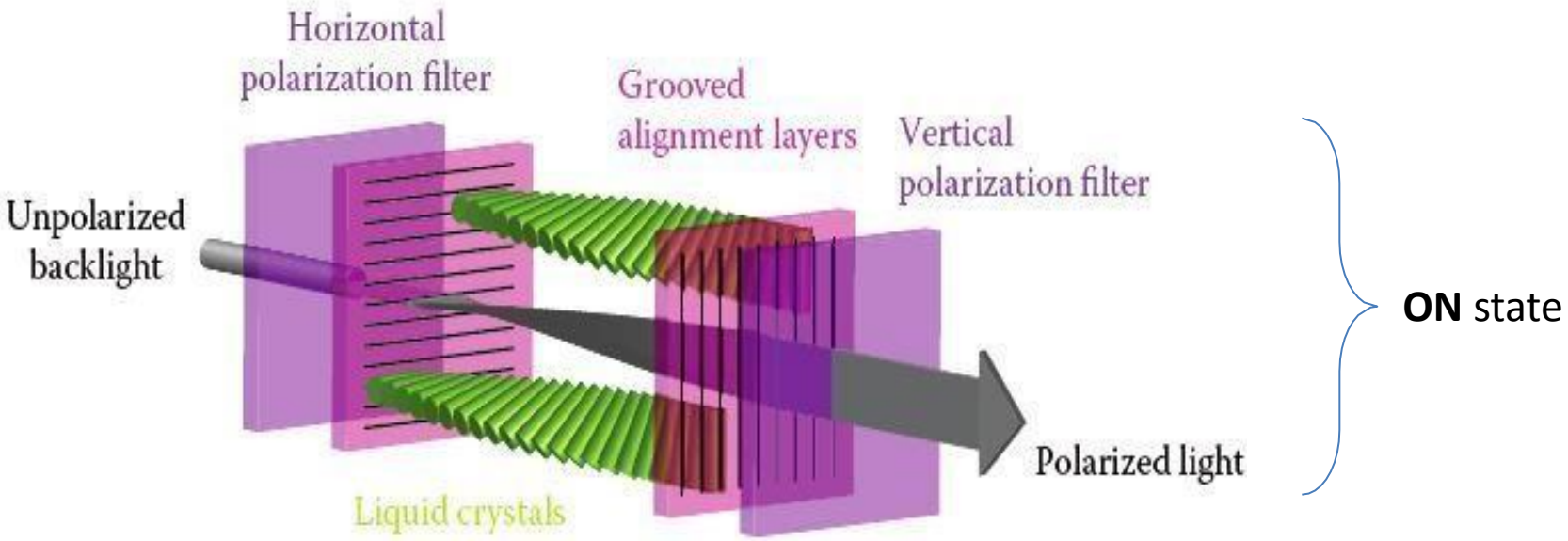


OFF state

Credit: Fundamentals of Computer Graphics 3rd Edition by Peter Shirley, Steve Marschner | http://www.cs.cornell.edu/courses/cs4620/2019fa/

Transmissive Displays (2/3)

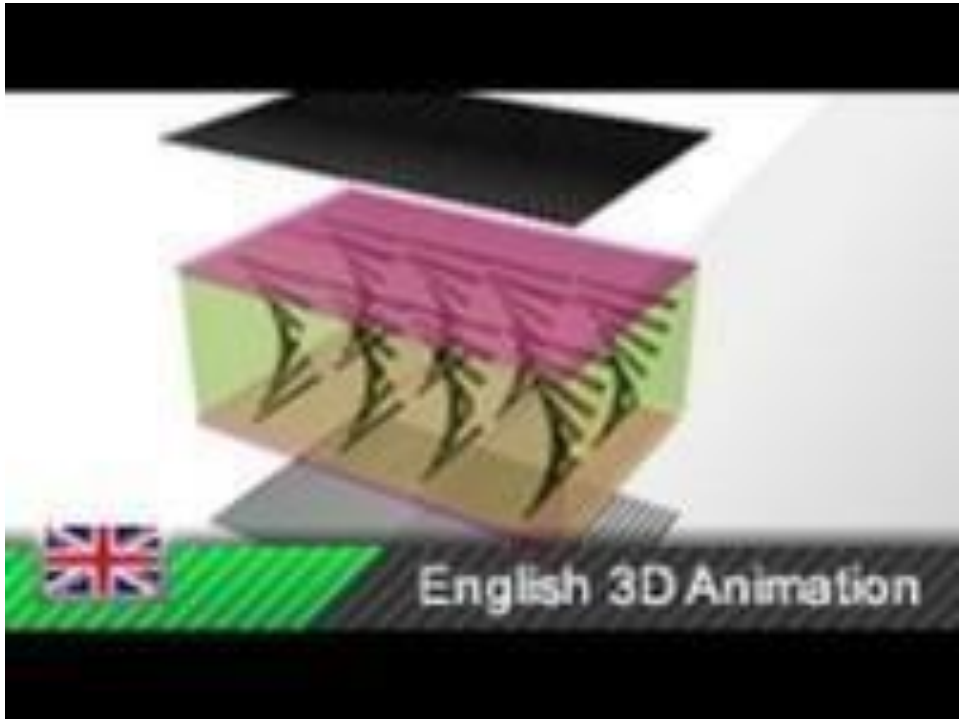
- Degree of rotation ↔ applied voltage



Credit: Fundamentals of Computer Graphics 3rd Edition by Peter Shirley, Steve Marschner | http://www.cs.cornell.edu/courses/cs4620/2019fa/

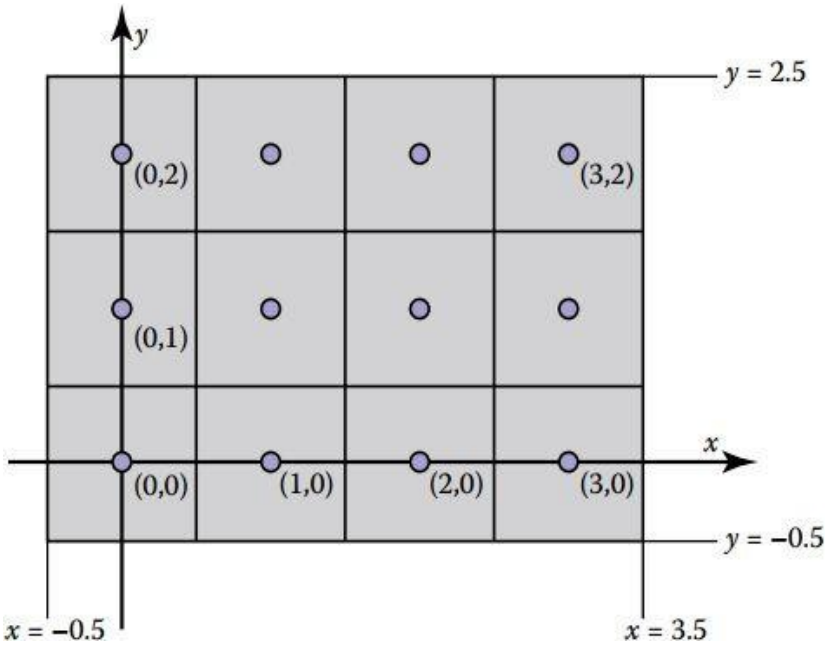
Transmissive Displays (3/3)

youtu.be/k7xGQKpQAWw?t=77



Pixel Values (1/1)

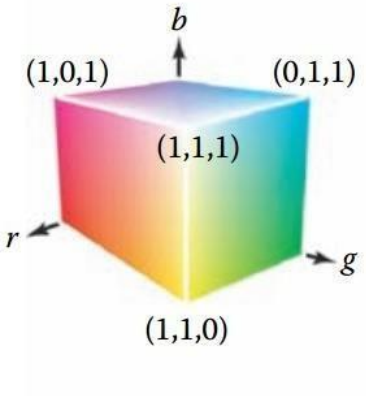
- Coordinate system for raster screen:
  - Convention:



Credit: Fundamentals of Computer Graphics 3rd Edition by Peter Shirley, Steve Marschner | http://www.cs.cornell.edu/courses/cs4620/2019fa/

RGB Color (1/1)

black = (0, 0, 0), red = (1, 0, 0), green = (0, 1, 0),
blue = (0, 0, 1), yellow = (1, 1, 0), magenta = (1, 0, 1),
cyan = (0, 1, 1), white = (1, 1, 1)



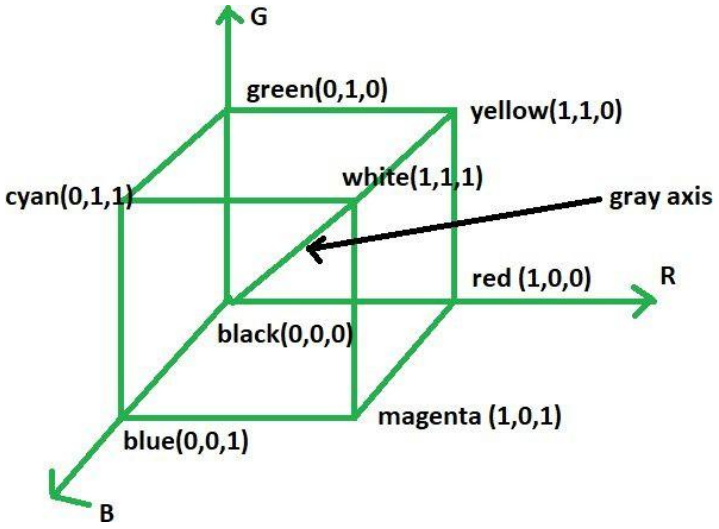


Image Source:
https://www.geeksforgeeks.org/computer-graphics-the-rgb-color-model/

Credit: Fundamentals of Computer Graphics 3rd Edition by Peter Shirley, Steve Marschner | http://www.cs.cornell.edu/courses/cs4620/2019fa/

Graphics Processing Unit

What is a GPU and Why Do We Need It?

- GPU stands for Graphics Processing Unit
- It as a super-specialized helper for your computer's main brain (the CPU)



Image Source: Internet

Real-World GPU Workloads

3D Graphics Rendering:

- Model Space to world space conversion
- Rotation, scaling, projection, rasterization, shading—each step parallelized across thousands of fragments or vertices.

Cryptocurrency Mining :

- Performing millions of SHA-256 hashes per second in parallel

Neural Network Training:

- Tensor Cores are designed to accelerate matrix multiply accumulate operations (D=A×B+C)

GPU vs CPU

CPU (Central Processing Unit)

- Designed with complex Control logic and large Cache memory (L1, L2, L3). It has fewer, more powerful ALUs (Arithmetic Logic Units).
- Optimized for serial operations – doing one task after another, very quickly
- Runs at higher clock speeds
- Low latency tolerance – it wants to finish a single task as fast as possible

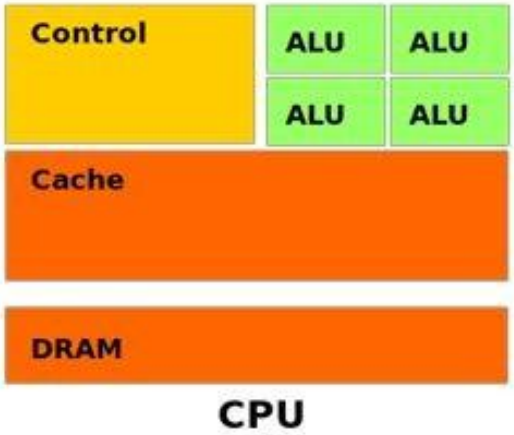


Image Source: Internet

GPU vs CPU

GPU (Graphics Processing Unit)

- Composed of a vast number of smaller, simpler ALUs
- Built for parallel operations – doing many simple tasks simultaneously
- Uses deep pipelines
- Achieves high throughput – gets a massive amount of work done in total
- Has high latency tolerance

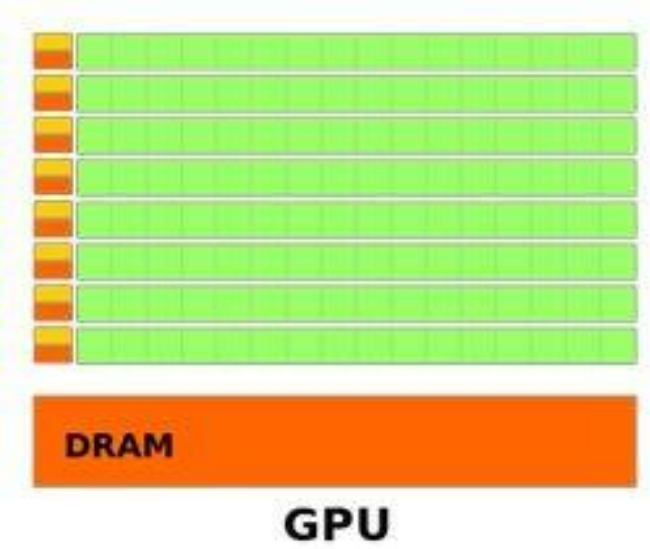


Image Source: Internet

Inside the GPU: Building Blocks

GPU is a complex chip with billions of tiny switches called transistors. It's organized like a company with different departments:

- **Graphics Processing Clusters (GPCs)**
  - **Streaming Multiprocessors (SMs)**
    - **CUDA**
    - **Tensor Cores**
    - **Ray-Tracing Cores**

Inside the GPU: Building Blocks

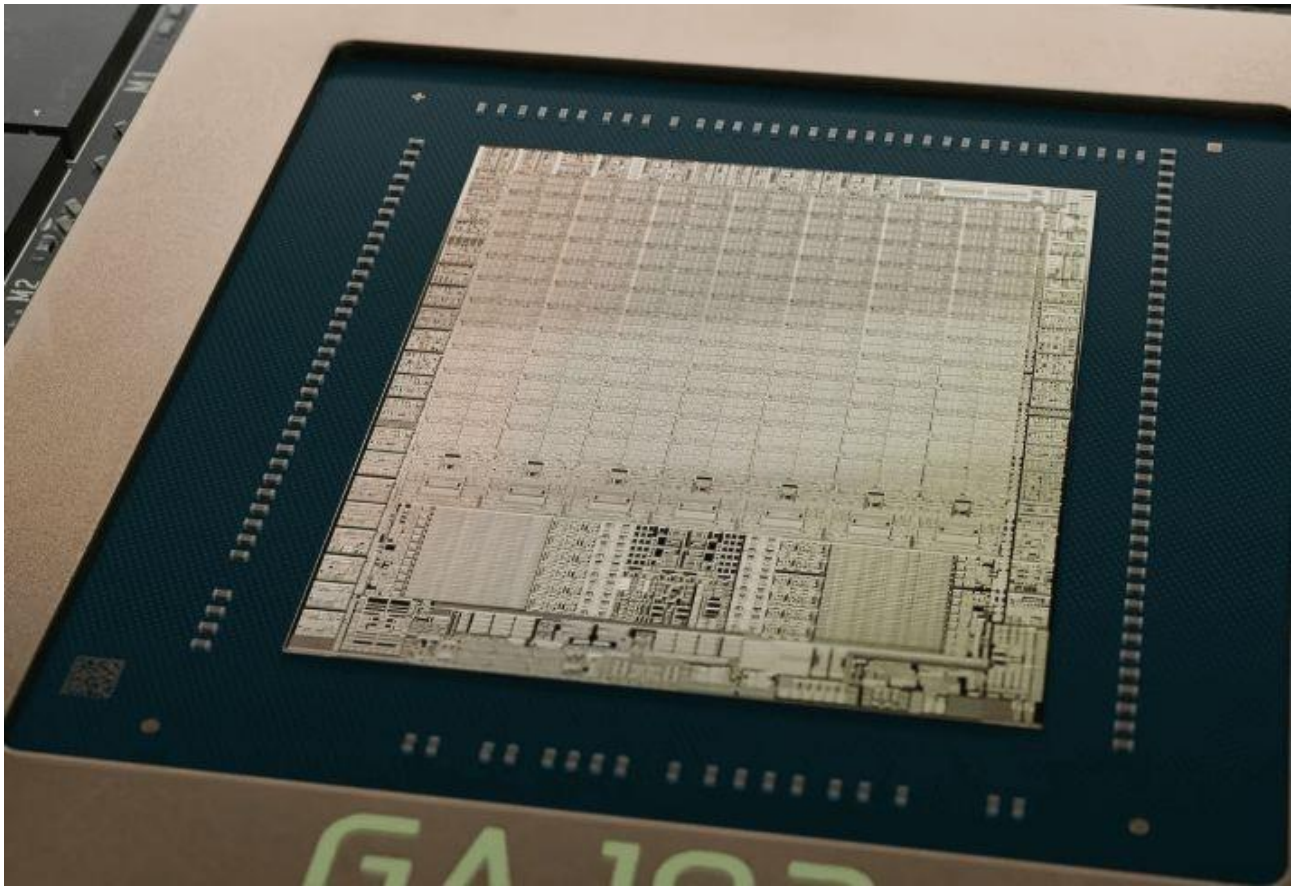


Fig: Close-up of the GA102 GPU's Silicon die revealing

Inside the GPU: Building Blocks

Graphics Processing Clusters (GPCs): These are like the main departments. The GA102 features 7 GPCs, each processing a subset of the rendering workload.

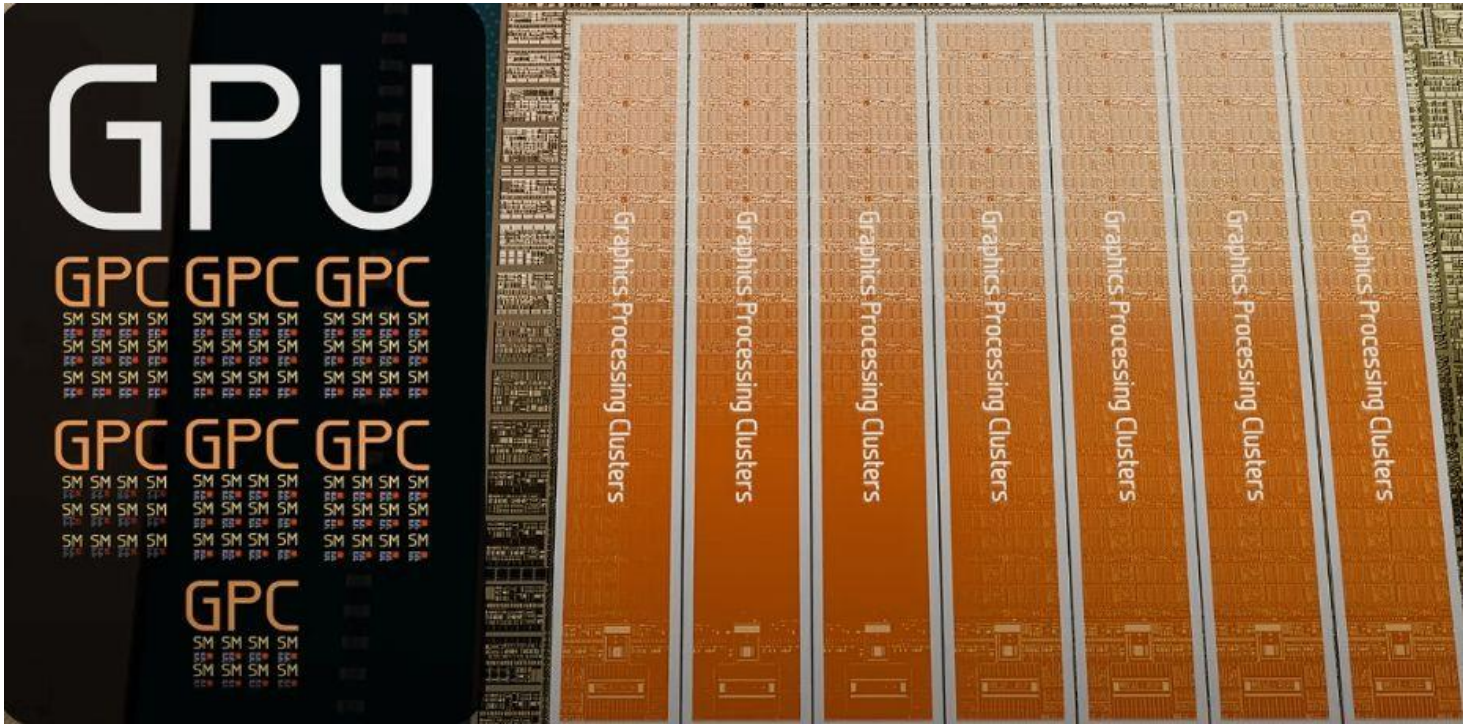


Fig: Diagram of seven Graphics Processing Clusters (GPCs) laid out on the GA102 die.

Image Source: Internet

Inside the GPU: Building Blocks

Streaming Multiprocessors (SMs): Inside each GPC, you find many SMs. Each GPC in GA102 houses 12 SMs, totaling 84 SMs on the die. SMs are the workhorses that dispatch and execute threads.

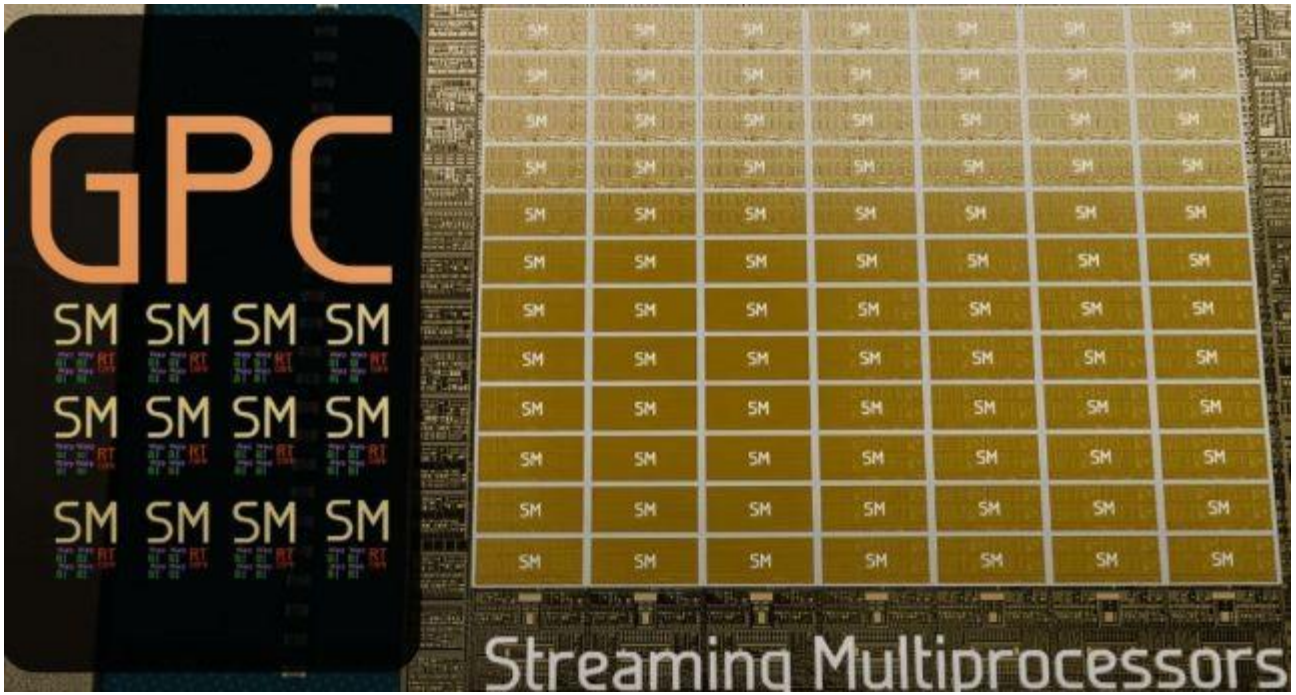


Fig: Grid view of the 84 Streaming Multiprocessors (SMs) within the GA102's Graphics Processing Clusters.

Image Source: Internet

Inside the GPU: Building Blocks

What's inside an SM?

CUDA (Shading) Cores: These are the most common workers. There are 10,752 CUDA Cores on GA102. They handle basic operations like add, multiply, and bitwise logic.

Tensor Cores: Specialized in matrix multiply-accumulate operations, essential for AI workloads. There is one Tensor Core per warp (totaling 336 on GA102).

Ray-Tracing Cores (RT Cores): Dedicated to processing ray-tracing acceleration structures and intersection tests for realistic lighting. There is one RT Core per SM (totaling 84 on GA102).

Image Source: Internet

Inside the GPU: Building Blocks

What's inside an SM?

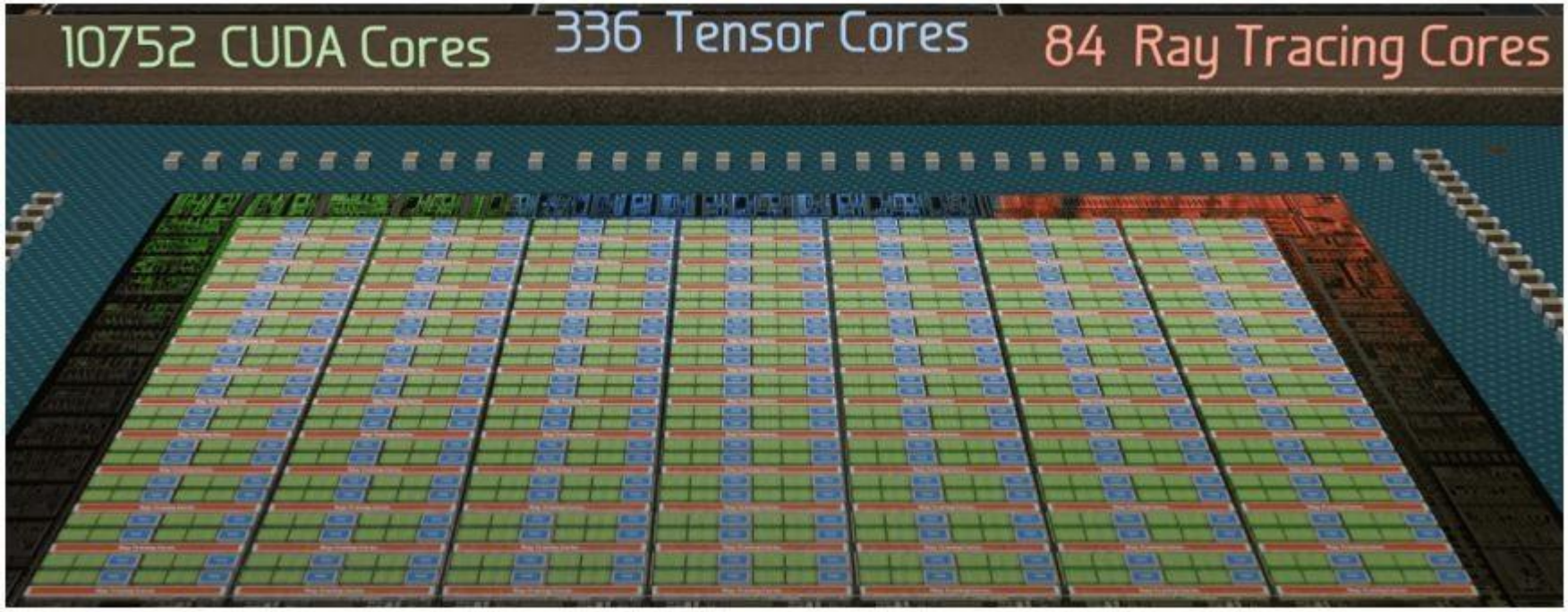


Fig: Color-coded layout showing 10,752 CUDA cores alongside 336 Tensor cores and 84 RT cores on the GA102 die.

Image Source: Internet

Inside the GPU: Building Blocks

How does it work?

Threads: Imagine each tiny calculation needed is a thread. Each thread is like a single instruction given to one CUDA core.

Thread Blocks: Threads are grouped into thread blocks. Threads within a block can work together and share data easily. Each thread block is usually handled by one SM.

Warps: Inside a thread block, threads are further grouped into warps (usually 32 threads). All threads in a warp execute the same instruction at the same time.

Inside the GPU: Building Blocks

How does it work?

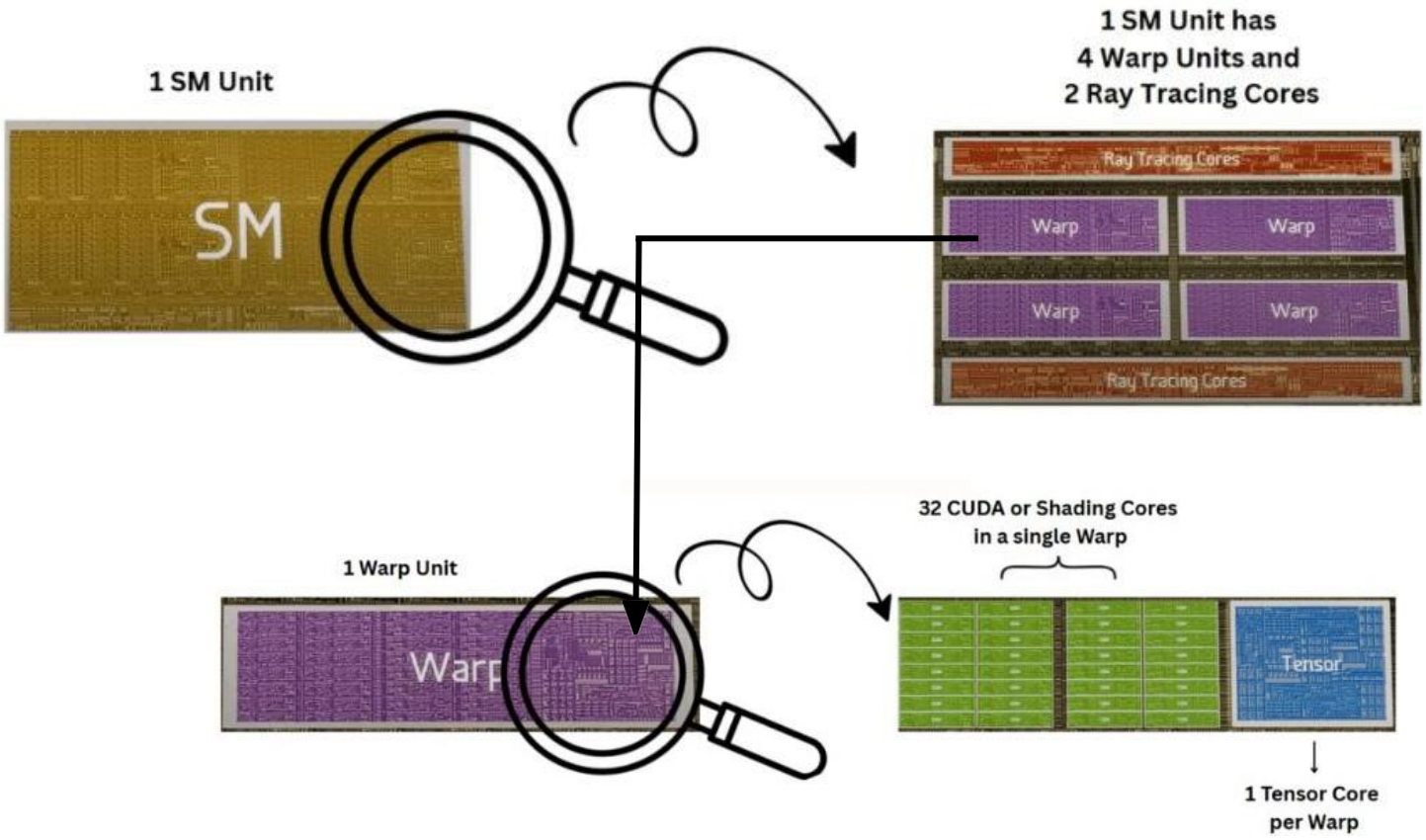


Fig: Zoomed view of one warp showing its 32 CUDA cores alongside the warp's single Tensor core in a single Multiprocessor

Image Source: Internet

Inside the GPU: Building Blocks

How does it work?

GPUs excel at embarrassingly parallel tasks, where identical operations are applied independently across vast datasets (e.g., 3D vertex transformations, pixel filters, neural network matrix operations).

Inside the GPU: Building Blocks

How does it work?

SIMD (Single Instruction, Multiple Data):

In older GPUs, when a warp (team of 32) got an instruction, all 32 threads had to do it at the exact same time. If one thread finished early or had to wait, all the others had to wait too.

SIMT (Single Instruction, Multiple Threads):

This is how modern NVIDIA GPUs work. When a warp gets an instruction, all 32 threads still get the same instruction, but each thread can work at its own pace. They don't have to be perfectly in sync.

Inside the GPU: Building Blocks

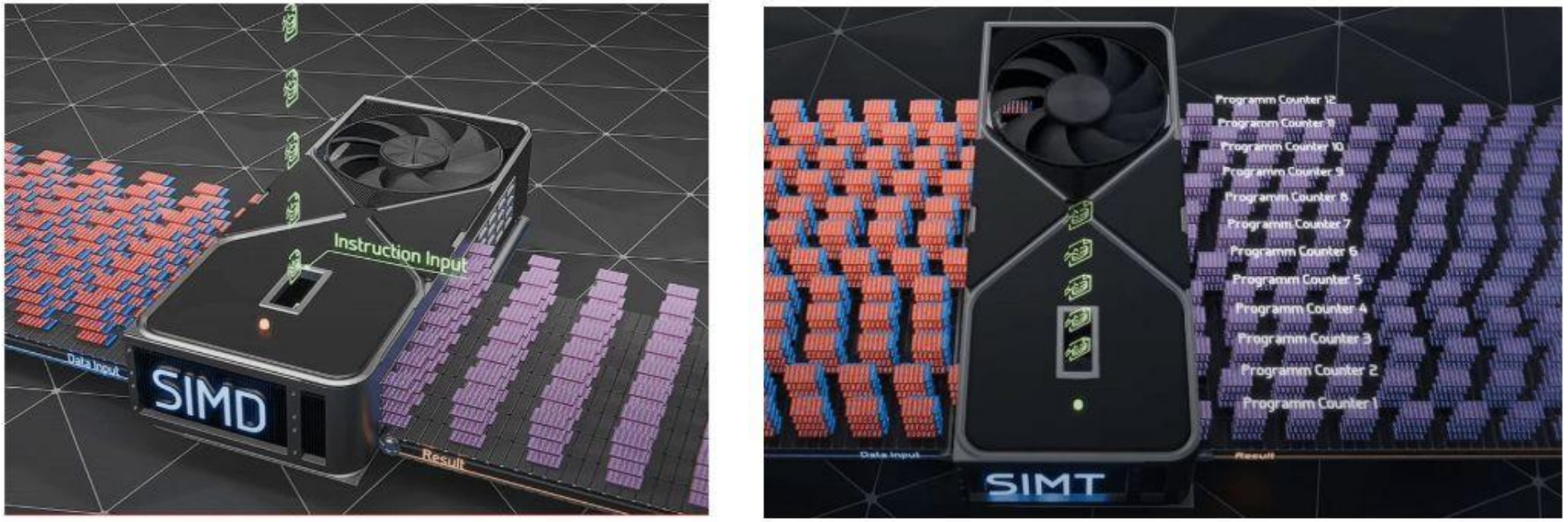


Fig: (Left) Early SIMD model: a single instruction broadcast to all lanes in lockstep. (Right) SIMT model with each thread maintaining its own program counter for flexible execution.

47mage Source: Internet

Inside the GPU: GPU Memory

GPUs are data-hungry machines and need a constant supply of data to keep thousands of cores busy. They have a specialized memory system:

Registers: Super-fast, on-chip memory locations private to each thread within an SM, used for immediate data.

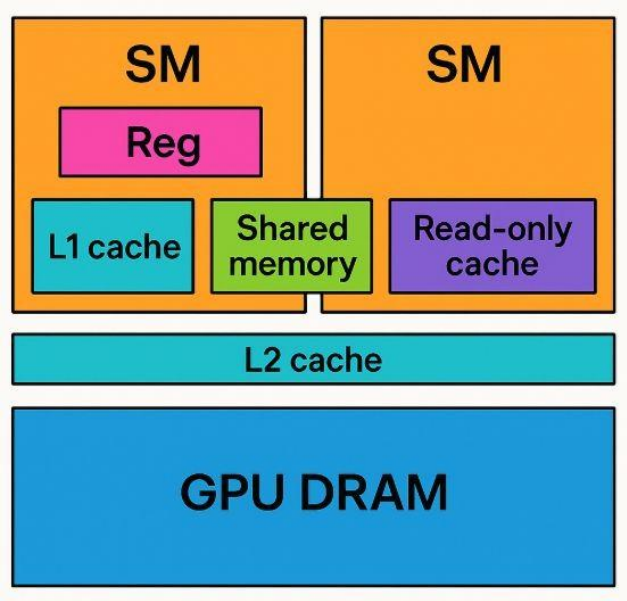


Fig: GPU Memory Hierarchy

Inside the GPU: GPU Memory

L**1 Cache / Shared Memory:** A small, very fast on-chip memory private to each SM. The size can vary (e.g., 128KB for V100, 192KB for A100, 256KB for H100). It's shared among threads in a thread block and ideal for frequently accessed data.

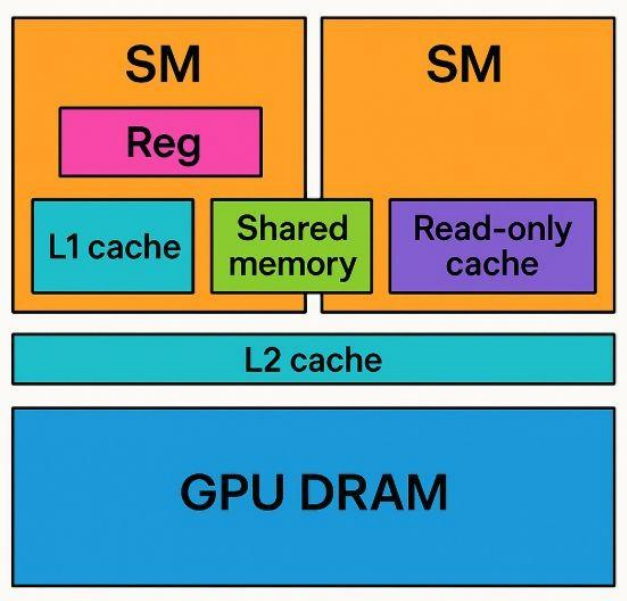


Fig: GPU Memory Hierarchy

Inside the GPU: GPU Memory

L2 Cache: A larger, on-chip cache shared across all SMs. It keeps data read/written from/to device memory to service SM requests. (e.g., 6.144 MB for V100, 40 MB for A100, 60 MB for H100).

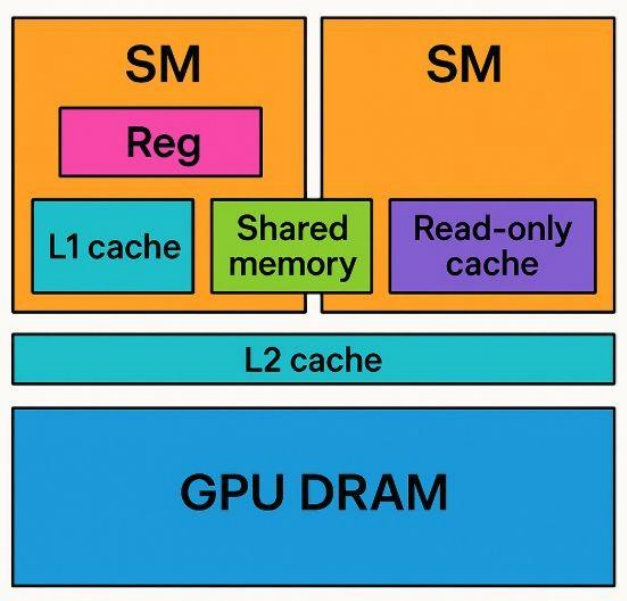


Fig: GPU Memory Hierarchy

Inside the GPU: GPU Memory

Device Memory: This is the main, large memory off-chip. It's made of different generations of High Bandwidth Memory (HBM2, HBM2e, HBM3). Provides significantly higher memory bandwidth compared to CPU DRAM (e.g., up to 4000 GB/s for HBM3 in H100). It stores large game textures, 3D models, and big datasets for AI.

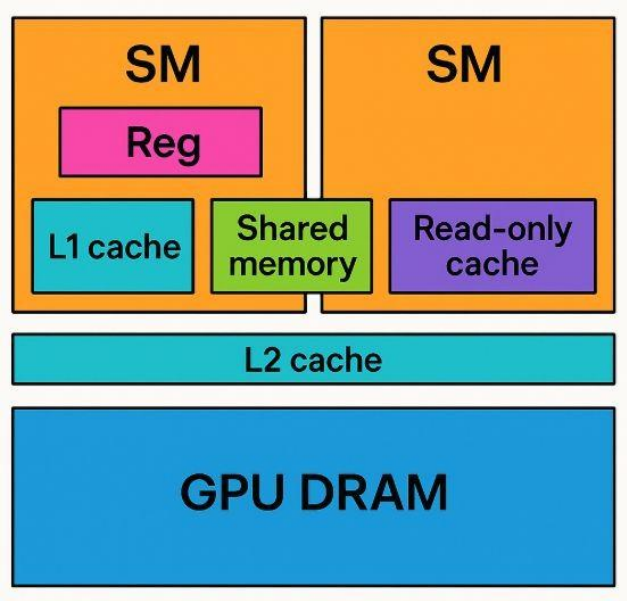


Fig: GPU Memory Hierarchy

Practice Question 1

Consider an image with resolution of 1500 x 1500. Each pixel in the image can hold up to 8 bit of data.

a) What is the file size of the image in MB?

b) If the image is compressed with a compression ration of 1.5. What is the file size of the compressed image?

Practice Question 1

Consider an image with resolution of 1500 x 1500. Each pixel in the image can hold up to 8 bit of data.

a) What is the file size of the image in MB?

Solution:

Total number of pixels in the image = 1500 x 1500 = 2250000

Each pixel can hold 8 bit data.

So, the size of the original image = 2250000 x 8 = 1800,0000 bits

= 18000,000 / 8 = bytes = 2250000 bytes = ?? MB

Practice Question 1

Consider an image with resolution of 1500 x 1500. Each pixel in the image can hold up to 8 bit of data.

b) If the image is compressed with a compression ration of 1.5. What is the file size of the compressed image?

Solution:

Compression ratio = original image size / compressed image size

Compressed image size = original image size / compression ratio

= 2.25 MB / 1.5 = 1.5 MB

Practice Question 2

- Consider an image IMG with the pixel values shown in the figure. Each pixel in the image can hold up to 8 bits of data. Find the file size of the image in KB. Also, determine the compression ratio of the image, if the image is compressed by applying run-length compression.

0	0	0	1	1	1	2
0	0	0	0	0	1	1
2	2	2	2	0	1	1
2	2	3	3	3	3	3
3	3	3	3	3	1	1
3	3	3	2	2	0	0

Further Reading

- Fundamentals of Computer Graphics, 4th Edition - Chapter 3
- https://www.adobe.com/uk/creativecloud/photograp hy/discover/lossy-vs-lossless.html
- https://learnopencv.com/modern-gpu-architecture-explained/
- https://www.youtube.com/watch?v=h9Z4oGN89MU&ab_channel=BranchEducation

Thank You